

Tutorial 00 — Part 4

NumPy & Matplotlib

Arrays, random numbers, and plots

Shuvam Banerji Seal

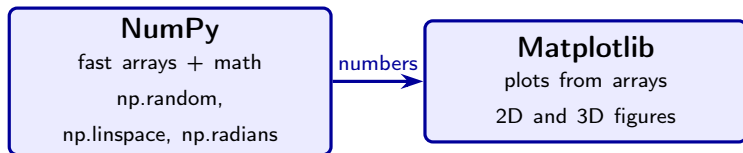
CH4114 — Molecular Simulation

14 August 2026

Course Instructor: Dr. Susmita Roy · TA: Shuvam Banerji Seal

Repository: `CH4114_Molecular_Simulation_Tutorials`

The two pillars of scientific Python



- Every simulation this semester is built on NumPy arrays.
- Every result is understood through a Matplotlib figure.

The standard imports

```
import numpy as np      import matplotlib.pyplot as plt
```

NumPy arrays — the basic container

```
a = np.array([1, 2, 3, 4, 5])
a.shape      # (5,)
a.dtype      # int64
a[0]         # 1
a[-1]        # 5
a[1:4]       # [2 3 4]
```

```
m = np.array([[1, 2], [3, 4]])
m.shape      # (2, 2)
m[0, 1]      # 2
```

- An array is like a list, but **fast and math-aware**.
- `shape` = size per dimension.
- `dtype` = type of the elements.
- Indexing and slicing work exactly like lists.

The big five array creators

Function	What it does
<code>np.array(...)</code>	array from a Python list
<code>np.arange(a, b, s)</code>	evenly spaced <i>integers</i>
<code>np.linspace(a, b, n)</code>	n evenly spaced values, inclusive
<code>np.zeros(s) / np.ones(s)</code>	array filled with 0 or 1
<code>np.zeros_like(x)</code>	zeros with the same shape as x

`np.linspace` is the most-used NumPy function in this course

`x = np.linspace(0, 2*np.pi, 100)` — the x-values of every plot you will make.

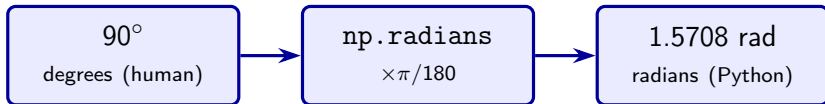
Random numbers: np.random

Function	What it gives you
<code>np.random.rand(n)</code>	n values uniform in $[0, 1)$
<code>np.random.randint(a, b, n)</code>	n random integers in $[a, b)$
<code>np.random.normal(m, s, n)</code>	n Gaussian values
<code>np.random.uniform(a, b, n)</code>	n values uniform in $[a, b)$
<code>np.random.choice(x, n)</code>	n random picks from x

Always set the seed first

`np.random.seed(42)` — makes “random” results **reproducible**. This is the heart of Monte Carlo simulation.

Angles: `np.radians` and `np.degrees`



- `np.sin`, `np.cos`, `np.tan` expect **radians**.
- `np.degrees(rad)` converts back.
- Useful constants: `np.pi`, `np.sqrt`, `np.exp`, `np.abs`.

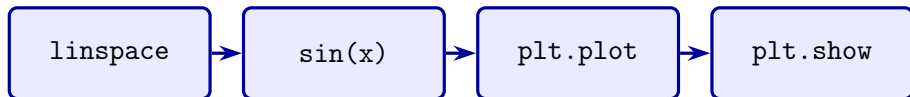
Math on arrays — no loops needed

The Lennard-Jones potential (you will use this pattern constantly)

```
r = np.linspace(0.3, 1.0, 50)      # 50 distances in nm
sigma, epsilon = 0.34, 0.997      # LJ parameters
sr = sigma / r                     # divide EVERY element by r
lj = 4 * epsilon * (sr**12 - sr**6) # formula on the whole array
lj.min(), lj.mean(), lj.sum()      # quick statistics
```

- Generate a range with `linspace`, apply a formula, get an array back.
- This is exactly how simulation analysis code is written.

Matplotlib — your first 2D plot



```
x = np.linspace(0, 2*np.pi, 100)
y = np.sin(x)
plt.plot(x, y)
plt.title("Sine wave"); plt.xlabel("angle (rad)"); plt.ylabel("sin(x)")
plt.grid(True); plt.show()
```


Colors, styles, and legends

Color ways	Examples
named color	"red", "crimson", "forestgreen"
hex code	"#2E86AB", "#FF5733"
RGB tuple	(0.1, 0.4, 0.9)
single letter	'r', 'g', 'b', 'k'

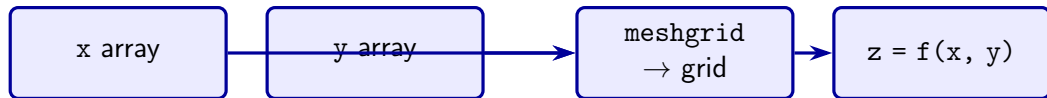
```
plt.plot(x, np.sin(x), color="crimson", label="sin")
plt.plot(x, np.cos(x), color="#2E86AB", linestyle="--", label="cos")
plt.scatter(x[::10], np.sin(x[::10]), color="green", s=30)
plt.legend()      # shows the labels
```

Subplots — several panels in one figure

```
fig, axes = plt.subplots(1, 2,  
                          figsize=(10, 4))  
  
axes[0].plot(x, np.sin(x))  
axes[0].set_title("sin")  
axes[1].plot(x, np.cos(x))  
axes[1].set_title("cos")  
plt.tight_layout()  
plt.show()
```

- `subplots(rows, cols)` returns a figure and axes.
- Draw on each axes:
`axes[0].plot(...)`.
- Inside an axes use `set_title`, `set_xlabel`.
- Grid: `subplots(2, 2) → axes[0,0], axes[0,1], ...`

3D surface plots



```
ax = fig.add_subplot(111, projection="3d")
X, Y = np.meshgrid(u, u)           # grid of (x, y) pairs
Z = np.exp(-(X**2 + Y**2))         # value at every grid point
ax.plot_surface(X, Y, Z, cmap="viridis")  # colored surface
```

- Colormaps: "viridis", "plasma", "coolwarm", "magma".

3D scatter and rotating with your mouse

```
np.random.seed(7)
n = 50
xs = np.random.rand(n)
ys = np.random.rand(n)
zs = np.random.rand(n)
ax.scatter(xs, ys, zs,
           s=50, c=zs, cmap="plasma")
```

Mouse controls (any 3D plot)

- **Drag** left button → rotate
 - **Right-drag** → zoom
 - **Scroll** → zoom in/out
-
- Script: `plt.show()` opens a rotatable window.
 - Jupyter: `%matplotlib widget` (needs `ipympl`).

Scripts, notebooks, and check yourself

Run the companion code

```
uv run python examples/numpy_matplotlib/numpy_basics.py
uv run python examples/numpy_matplotlib/matplotlib_plots.py
```

Check yourself

- 1 `np.arange(0,10,2)` vs `np.linspace(0,10,2)` — difference?
- 2 Why set `np.random.seed(...)` before using `np.random`?
- 3 Which function converts degrees to radians? Why is it needed for `np.sin`?
- 4 What does `np.zeros_like(x)` return?
- 5 How do you make a 1×3 grid of subplots? A 3D axes?